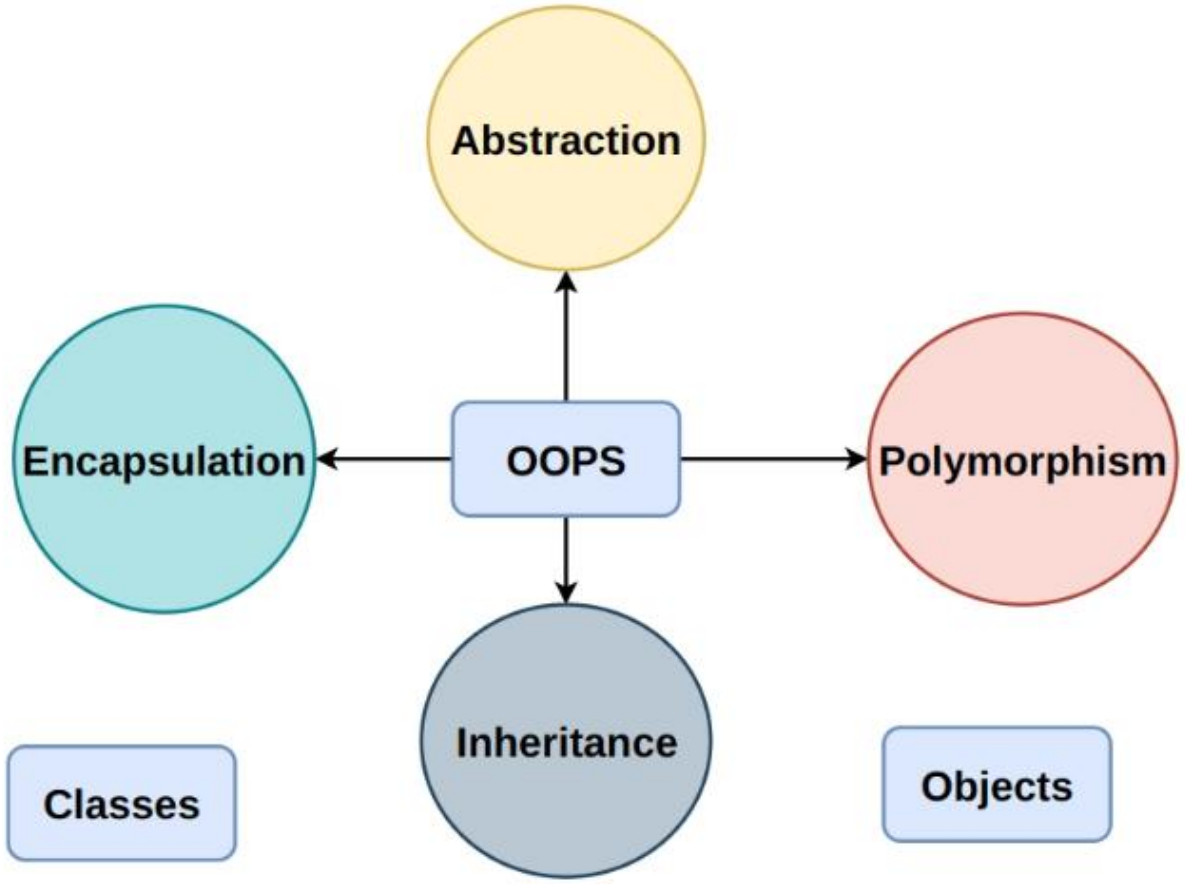
Module 3

Features of OOP

Object-oriented programming, or OOPs, is a programming model which breaks down a problem in terms of classes and objects.

OOPs allows the creation of several instances of a class called objects, hence facilitating code reuse.



Class

- It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class.
- The compiler does not allocate memory whenever you define a class.

Object

- An Object is an instance of a Class.
- When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

class	objects
Fruit	Apple
	Banana
	Mango

Abstraction

- Object Oriented Programming abstraction exposes only the essential information of an object to the user and hides the other details.
- We can implement abstraction using classes that group the data members and function together.
- Inside classes, we can choose the access specifiers for its members to control how they are visible to the outside world.

Inheritance

- It allows a class to inherit the properties and methods of another class called the parent class, the base class, or the super-class.
- The class that inherits is called the child class or sub-class.

Polymorphism

- The word polymorphism means to have many forms. So, by using polymorphism, we can add different meanings to a single component.
- There are two types of polymorphism:
  - Compile-time polymorphism – Method overloading
  - Run-time polymorphism – Method overriding

Encapsulation

- Encapsulation means enclosing the data/variables and the methods for manipulating the data into a single entity called a class.
- It helps to hide the internal implementation of the functions and state of the variables, promoting abstraction.

I/O in C and C++

```c
#include <stdio.h>

int main() {
    int intVal;
    float floatVal;
    char charVal;
    char strVal[100];

    // Input operations
    printf("Enter an integer: "); scanf("%d", &intVal);
    printf("Enter a float: "); scanf("%f", &floatVal);
    printf("Enter a character: ");
    scanf(" %c", &charVal); // Space before %c to
                            // consume newline char
    printf("Enter a string: ");
    scanf(" %[^\n]", strVal); // Read string with spaces
```

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int intVal;
    float floatVal;
    char charVal;
    string strVal;
    // Input operations
    cout << "Enter an integer: ";
    cin >> intVal;
    cout << "Enter a float: ";
    cin >> floatVal;
    cout << "Enter a character: ";
    cin >> charVal;
    cin.ignore(); // Ignore the newline char left in the
                  //buffer
    cout << "Enter a string: ";
    getline(cin, strVal); // Read string with spaces
}
```

I/O in C and C++

```
    // Output operations
    printf("\nYou entered:\n");
    printf("Integer: %d\n", intVal);
    printf("Float: %f\n", floatVal);
    printf("Character: %c\n", charVal);
    printf("String: %s\n", strVal);

    return 0;
}
```

```
    // Output operations
    cout << "\nYou entered:\n";
    cout << "Integer: " << intVal << endl;
    cout << "Float: " << floatVal << endl;
    cout << "Character: " << charVal << endl;
    cout << "String: " << strVal << endl;

    return 0;
}
```

```
cout << "Integer: " << intVal << endl << "Float: " << floatVal << endl << "Character: "
<< charVal << endl << "String: " << strVal << endl;
```

C and C++

```c
#include <stdio.h>

int main() {
    float num1, num2, sum;

    // Input two numbers
    printf("Enter first number: ");
    scanf("%f", &num1);
    printf("Enter second number: ");
    scanf("%f", &num2);

    // Calculate sum
    sum = num1 + num2;

    // Output result
    printf("Sum: %f\n", sum);

    return 0;
}
```

```cpp
#include <iostream>
using namespace std;

int main() {
    float num1, num2, sum;

    // Input two numbers
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;

    // Calculate sum
    sum = num1 + num2;

    // Output result
    cout << "Sum: " << sum << endl;
    // cout << "Sum: " << sum << “\n”;
    return 0;
}
```

Creating a Class

```
class className{
    Access specifier // can be private, protected, public
    Data members  // variables
    Member Functions() // methods to access data members
}; // Class ends with a semicolon
```

Declaring objects

- Syntax: `className objectName;`
- When a class is defined, only the specifications or the blueprint for the object is defined, no memory is allocated.
- In order to use the data and access the functions defined inside a class, we need an object.
- In order to access the data members and member functions of a class, the dot ('.') operator is used along with the object name.

Access Specifiers

- Access Specifiers are used to implement data hiding.
- public
- private - default
- protected

Access Specifier	Accessible in Class	Accessible in Derived Class	Accessible in main()
private	✓	✗	✗
protected	✓	✓	✗
public	✓	✓	✓

- Public
- All the class members declared under this are available to everyone, that is, they can be accessed by other classes and functions too. The public members are directly accessible using the dot operator with the object of the class from anywhere in the program.
- Private
- All the class members declared under private can be accessed only by the member functions inside the class, they can not be accessed by any object or function outside the class. The member functions and the friend functions are allowed to access them.
- If we don't specify any access modifiers for the members inside the class, then by default, the access modifier for members is set as Private.
- Protected
- The protected access modifier is similar to the private access modifier in the sense that it can’t be accessed outside of its class unless with the help of a friend class.
- The difference is that the class members declared as Protected can be accessed by any subclass (derived class) of that class as well.

```
// Define a class named 'Person’
class Person {
public:    // Data members
string name;
int age;
void introduce()   {   // Member function to introduce the person
cout << "Hi, my name is " << name << " and I am "  << age << " years old." << endl;
}
};
int main(){
    Person person1; // Create an object of the Person class
    person1.name = "Alice"; // accessing data members
    person1.age = 30;
    person1.introduce(); // Call the introduce member method
    return 0;
}
```

```
#include <iostream>
using namespace std;
class Adder {
private:
   float num1, num2, sum;
public:
      void getInput() {
      cout << "Enter first number: ";
      cin >> num1;
      cout << "Enter second number: ";
      cin >> num2;
   }
      void calculateSum() {
      sum = num1 + num2;
   }
      void displayResult() {
      cout << "Sum: " << sum << endl;
   }
};
```

```
int main() {
   Adder obj1;  // Creating an object of class Adder
   obj1.getInput();      // Taking input
   obj1.calculateSum();  // Performing addition
   obj1.displayResult(); // Displaying result

   Adder obj2;  // Creating an object of class Adder
   obj2.getInput();      // Taking input
   obj2.calculateSum();  // Performing addition
   obj2.displayResult(); // Displaying result

   return 0;
}
```

Scope Resolution Operator ::

```
#include <iostream>
using namespace std;
class Adder {
private:
float num1, num2, sum;
public:
void getInput(); // Function prototype
void calculateSum(); // Function prototype
void displayResult(); // Function prototype
};
void Adder::getInput() {
cout << "Enter first number: ";
cin >> num1;
cout << "Enter second number: ";
cin >> num2;
}
void Adder::calculateSum() {
sum = num1 + num2;
}
```

```
void Adder::displayResult() {
cout << "Sum: " << sum << endl;
}
int main() {
   Adder obj;  // Creating an object of class Adder

   obj.getInput();      // Taking input
   obj.calculateSum();  // Performing addition
   obj.displayResult(); // Displaying result

   return 0;
}
```

public

```
#include <iostream>
using namespace std;
class Adder {
public:
   float num1, num2; // Public data members
   void getInput() {
      cout << "Enter first number: ";
      cin >> num1;
      cout << "Enter second number: ";
      cin >> num2;
   }
   void displayResult() {
      cout << "Sum: " << num1 + num2 << endl;
   }
};
```

```
int main() {
   Adder obj;
   obj.getInput();

   // Directly accessing public members
   cout << obj.num1 + obj.num2 << endl;
  cout << "Input values are "<< obj.num1 << " and " <<
obj.num2 << endl;
   obj.displayResult();

   return 0;
}
```

```
Enter first number: 12
Enter second number: 34
Input values are 12 and 34
Sum: 46
```

private

```
#include <iostream>

using namespace std;

class Adder {
private:
   float num1, num2; // Private members

public:
   void getInput() {
      cout << "Enter first number: ";
      cin >> num1;
      cout << "Enter second number: ";
      cin >> num2;
   }
   void displayResult() {
      cout << "Sum: " << num1 + num2 << endl;
   }
};
```

```
int main() {
   Adder obj;
   obj.getInput();

   // error because num1 and num2 are private
   // cout << "Input values are "<< obj.num1 << " and " <<
obj.num2 << endl;
   obj.displayResult();

   return 0;
}
```

protected

```
#include <iostream>
using namespace std;

class Adder {
protected:
   float num1, num2; // Protected members
public:
   void getInput() {
      cout << "Enter first number: ";
      cin >> num1;
      cout << "Enter second number: ";
      cin >> num2;
   }
};

class DerivedAdder : public Adder {
public:
   void displayResult() {
      cout << "Sum: " << num1 + num2 << endl;
     // Accessible in derived class
   }
};
```

```
int main() {
   DerivedAdder obj;
   obj.getInput();

   // error because num1 and num2 are protected
   // cout << "Input values are "<< obj.num1 << " and " <<
obj.num2 << endl;

   obj.displayResult();

   return 0;
}
```

Inline function

- An inline function in C++ is a function that is expanded in line when it is called.
- Instead of making a normal function call, the compiler replaces the function call with the actual function code, which can improve performance by eliminating the overhead of function calls.
- The function is prefixed with the inline keyword.
- Avoids Function Call Overhead: Instead of jumping to a separate memory location, the function's code is inserted directly at the call site.
- Best for Small Functions: Using inline for large functions can lead to code bloat, making the executable larger.
- Compiler May Ignore inline: The compiler may decide not to inline a function if it deems it inefficient.

Inline function

```
#include <iostream>

using namespace std;

inline int add(int a, int b) {
    return a + b;
}

int main() {
    cout << "Sum: " << add(5, 3) << endl;
    return 0;
}
```

Here, add(5, 3) will be replaced directly by 5 + 3 during compilation.

Constructors

- A **constructor** is a special member function in C++ that is automatically called when an object of a class is created.
- It is used to initialize objects.
- The constructor has the same name as the class and does not have a return type.
- **Types of Constructors in C++**
  1. Default Constructor (No parameters)
  2. Parameterized Constructor
  3. Copy Constructor

Default Constructor

- A constructor that takes no parameters and initializes class members with default values.

```
#include <iostream>
using namespace std;

class Car {
public:
   string brand;

   // Default Constructor
   Car() {
      brand = "Toyota";
      cout << "Default Constructor Called!" << endl;
   }
   void display() {
      cout << "Brand: " << brand << endl;
   }
};
```

```
int main() {
   Car c1;  // Default constructor is automatically called
   c1.display();
   return 0;
}
```

Parameterized Constructor

- A constructor that takes arguments to initialize class members.

```
#include <iostream>
using namespace std;

class Car {
public:
   string brand;

   // Parameterized Constructor
   Car(string b) {
      brand = b;
      cout << "Parameterized Constructor Called!" << endl;
   }
   void display() {
      cout << "Brand: " << brand << endl;
   }
};
```

```
int main() {
   Car c1("Tesla");  // Pass argument to constructor
   c1.display();
   return 0;
}
```

Copy Constructor

- A constructor that creates a new object by copying an existing object.

```
#include <iostream>
using namespace std;
class Car {
public:
   string brand;
      // Parameterized Constructor
   Car(string b) {
      brand = b;
   }
   // Copy Constructor
   Car(Car &obj) {
      brand = obj.brand;
      cout << "Copy Constructor Called!" << endl;
   }
   void display() {
      cout << "Brand: " << brand << endl;
   }
};
```

```
int main() {
 Car c1("BMW");
   Car c2 = c1;  // Copy constructor is called
   Car c3(c2); // Copy constructor is called
   c1.display();
   c2.display();
   c3.display();
   c1.brand="KIA";
   c2.brand="AUDI";
   c1.display();
   c2.display();
   c3.display();
   return 0;}
```

```
Copy Constructor Called!
Copy Constructor Called!
Brand: BMW
Brand: BMW
Brand: BMW
Brand: KIA
Brand: AUDI
Brand: BMW
```

Destructor

- A destructor is used to clean up resources when an object goes out of scope. It has the same name as the class but starts with a tilde ~

```
#include <iostream>
using namespace std;

class Car {
public:
   string brand;

   Car(string b) {
      brand = b;
   }

   ~Car() {
      cout << "Destructor Called for " << brand << endl;
   }
};
```

```
int main() {
   Car c1("Ford");
   return 0;
}
```

```
Destructor Called for Ford
```

Destructor

```
#include <iostream>
using namespace std;

class Car {
public:
   string brand;

   Car(string b) {
      brand = b;
   }

   Car()
   {
      brand="---";
   }

   ~Car() {
      cout << "Destructor Called for " << brand << endl;
   }
};
```

```
int main() {
   Car c1("Ford");
   {
      Car c2;
   }
   return 0;
}
```

```
Destructor Called for ---
Destructor Called for Ford
```

```
int main() {
   Car c1("Ford");
   Car c2;
   return 0;
}
```

```
Destructor Called for ---
Destructor Called for Ford
```

```
int main() {
Car c2;
Car c1("Ford");
return 0;
}
```

```
Destructor Called for Ford
Destructor Called for ---
```

Static Data Members

- A static data member is a class member or class-level variable shared by all instances(objects) of the class.
- A static data member has only one copy for the entire class, which can be shared across all instances, which means that all objects of the class can access and modify the same value.

Declaration

```
class ClassName {
  public:
    static dataType staticMemberName;
};
```

Initialization

```
dataType ClassName::staticMemberName = initialValue;
```

Accessing Static Data Members

Using the Class Name (Recommended)
Using an Object (Not Recommended)

```
#include <iostream>
using namespace std;
class Student {
private:
   int id;
   string stuname;
   static string teachername;
public:
Student() {
   teachername="Guru";
}
Student(string tname) {
   teachername=tname;
}
  void getinput()   {
     cout << "Enter student ID and name ";
     cin >> id >> stuname;
  }
  void display()    {
     cout << id <<" " << stuname << " " << teachername << endl;
  }
  };
```

```
// Initialize static data member
string Student::teachername;

int main() {
   Student s1,s2;
   s1.getinput();
   s2.getinput();
   s1.display();
   s2.display();
   Student s3("Bejoy");
   s3.getinput();
   s1.display();
   s2.display();
   s3.display();
   return 0;
}
```

```
Enter student ID and name 1
Renu
Enter student ID and name 2
Ajmir
1 Renu Guru
2 Ajmir Guru
Enter student ID and name 3
Dev
1 Renu Bejoy
2 Ajmir Bejoy
3 Dev Bejoy
```

```
#include <iostream>
using namespace std;

class Student {
private:
   int id;
   string name;
public:
   static int studentCount; // Static data member
   // Constructor
   Student(string n) {
      name = n;
      id = ++studentCount;  // Assigns a unique ID based on static count
   }
   // Function to display student details
   void display() {
      cout << "Student ID: " << id << ", Name: " << name << endl;
   }
};
// Initialize static data member
int Student::studentCount = 0;
```

```
int main() {
   Student s1("Ajay");
   Student s2("Bob");
   Student s3("Chan");

   s1.display();
   s2.display();
   s3.display();

   // Accessing static data member
   cout << "Total Students: " << Student::studentCount ;

   return 0;
}
```

```
#include <iostream>
using namespace std;
class Student {
private:
   static int studentCount; // Static data member
   int id;
   string name;
public:
   // Constructor
   Student(string n) {
      name = n;
      id = ++studentCount;  // Assigns a unique ID based on static count
   }
   // Function to display student details
   void display() {
      cout << "Student ID: " << id << ", Name: " << name << endl;
   }
   //  function to get total student count
    int getStudentCount() {
      return studentCount;
   }
};
// Initialize static data member
int Student::studentCount = 0;
```

```
int main() {
   Student s1("Ajay");
   Student s2("Bob");
   Student s3("Chan");

   s1.display();
   s2.display();
   s3.display();

   // Calling function
   cout << "Total Students: " << s1.getStudentCount() << endl

   return 0;
}
```

Static Member Functions

- The static member functions are special functions used to access the static data members or other static member functions.
- A member function is defined using the static keyword.
- A static member function shares the single copy of the member function to any number of the class' objects.
- We can access the static member function using the class name or class' objects.
- If the static member function accesses any non-static data member or non-static member function, it throws an error.

```
#include <iostream>
using namespace std;
class Student {
private:
   static int studentCount; // Static data member
   int id;
   string name;
public:
   // Constructor
   Student(string n) {
      name = n;
      id = ++studentCount;  // Assigns a unique ID based on static count
   }
   void display() {// Function to display student details
      cout << "Student ID: " << id << ", Name: " << name << endl;
   }
  static int getStudentCount() {// Static function to get total student count
      return studentCount;
   }
};
// Initialize static data member
int Student::studentCount = 0;
```

```
int main() {
   Student s1("Ajay");
   Student s2("Bob");
   Student s3("Chan");

   s1.display();
   s2.display();
   s3.display();

   // Calling static function
   cout << "Total Students: “;
   cout << Student::getStudentCount() << endl;

   return 0;
}
```

```
#include <iostream>
using namespace std;
class Student {
private:
   static int studentCount; // Static data member
   int id;
   string name;
public:
   // Constructor
   Student(string n) {
      name = n;
      id = ++studentCount;  // Assigns a unique ID based on static count
   }
   // Function to display student details
   void display() {
      cout << "Student ID: " << id << ", Name: " << name << endl;
   }
   // Static function to get total student count
   static int getStudentCount() {
   //cout << "Student ID: " << id << ", Name: " << name << endl; Error
   return studentCount;
   }
};
// Initialize static data member
int Student::studentCount = 0;
```

```
int main() {
   Student s1("Ajay");
   Student s2("Bob");
   Student s3("Chan");

   s1.display();
   s2.display();
   s3.display();

   // Calling static function
   cout << "Total Students: " << Student::getStudentCount() <

   return 0;
}
```

Static Objects

- Static objects exist until the program terminates.
- Their destruction occurs automatically when main() ends.

```
Initial Student Count: 0
ID: 101, Name: Ajay

Inside destructor
ID: 102, Name: Bob
Total Students: 1

Inside destructor


...Program finished with exit code 0
Press ENTER to exit console.
```

```
    Student(int i, string n) {
        id = i;
        name = n;
        studentCount++; // Increment static counter
    }
    // Display student details
    void display() {
        cout << "ID: " << id << ", Name: " << name << endl;
    }
    // Static function to get student count
    static int getStudentCount() {
        return studentCount;
    }
```

```
// Destructor
    ~Student() {
    cout << "\nInside destructor\n";
     studentCount--; // Decrement student count when object is destroyed
    }
};

// Initialize static data member
int Student::studentCount = 0;
int main() {
    cout << "Initial Student Count: " << Student::getStudentCount() << end

    {
    Student s1(101, "Ajay");
    s1.display();
    }
    Student s2(102, "Bob");
    s2.display();
    cout << "Total Students: " <<     Student::getStudentCount() << endl;
    return 0;
}
```

```
Initial Student Count: 0
ID: 101, Name: Ajay
ID: 102, Name: Bob
Total Students: 2

Inside destructor
Bob

Inside destructor
Ajay
```

```
/ Static data member
public:
    // Constructor
    Student(int i, string n) {
        id = i;
        name = n;
        studentCount++; // Increment static counter
    }
    // Display student details
    void display() {
        cout << "ID: " << id << ", Name: " << name << endl;
    }
    // Static function to get student count
    static int getStudentCount() {
        return studentCount;
    }
```

```
// Destructor
    ~Student() {
    cout << "\nInside destructor\n";
     studentCount--; // Decrement student count when object is d
     cout<<this->name <<endl;
 }
};

// Initialize static data member
int Student::studentCount = 0;
int main() {
    cout << "Initial Student Count: " << Student::getStudentCount

    {
    static Student s1(101, "Ajay");
    s1.display();
    }
    Student s2(102, "Bob");
    s2.display();
    cout << "Total Students: " <<     Student::getStudentCount() <<
    return 0;
}
```

this pointer

- this pointer is an implicit pointer available in all non-static member functions of a class.
- It points to the current instance of the class that is calling the function.
- Cannot be used in static functions because they do not belong to any instance.

```
#include <iostream>
using namespace std;
class Student {
private:
   string sname;
public:
   void setName(string sname) {
      sname = sname;
   }
      void showName() {
      cout << "Student Name: " << sname << endl;
   }
};
int main() {
   Student s;
   s.setName("Alice");
   s.showName();
   return 0;
}
```

sname = sname; would assign the parameter to itself, not the instance variable.

```
#include <iostream>
using namespace std;
class Student {
private:
   string sname;
public:
   void setName(string sname) {
      this->sname = sname; // `this->name` refers to the instance variable
   }
      void showName() {
      cout << "Student Name: " << this->sname << endl;
   }
};
int main() {
   Student s;
   s.setName("Alice");
   s.showName();
   return 0;
}
```

It helps differentiate instance variables from function parameters when they have the same name.

Passing the Current Object as a Parameter

```
#include <iostream>
using namespace std;
class Car {
   string model;
public:
   Car(string model) { this->model = model; }
   void compare(Car other) {
     if (this->model == other.model)   // “this->model” is same as “model”, in this statement
     cout << "Both cars are the same model: " << model << endl;
     else
     cout << "Different car models: " << model << " vs " << other.model << endl;
   }
};
int main() {
   Car car1("Tesla Model X");
   Car car2("Tesla Model Y");
   Car car3("Tesla Model X");
   car1.compare(car2);
   car1.compare(car3);
   return 0;
}
```

```
Different car models: Tesla Model X vs Tesla Model Y
Both cars are the same model: Tesla Model X
```

Passing the Current Object as a Parameter

```
#include <iostream>
using namespace std;
class Car {
string model;
public:
   Car(string model) { this->model = model; }
   void compare(Car* other) {
      if (this->model == other->model)
            cout << "Both cars are the same model: " << model << endl;
      else
           cout << "Different car models: " << model << " vs " << other->model << endl;
   }
};
int main() {
    Car car1("Tesla Model X");
   Car car2("Tesla Model Y");
   Car car3("Tesla Model X");
   car1.compare(&car2);
   car1.compare(&car3);
   return 0;
```

```
Different car models: Tesla Model X vs Tesla Model Y
Both cars are the same model: Tesla Model X
```

Default Arguments in C++

- A default argument is a value provided for a parameter in a function declaration that is automatically assigned by the compiler if the calling function doesn’t provide a value for those parameters.
- If the value is passed for it, the default value is overridden by the passed value.

Default Arguments in C++

- The default values for parameters must be specified in the function declaration (or prototype). If a function is declared and defined separately, the default values must be in the declaration, not in definition.
- Once default arguments are defined, they cannot be modified in the function definition. If you try to change the default value in the definition, it will result in a compilation error.
- In a function with multiple parameters, default values must be provided from the rightmost parameter to the left. If a parameter has a default argument, all parameters to its right must also have default values.

```
#include <iostream>
using namespace std;

// Function with default arguments
void bookDetails(string title, string author = "Unknown", int pages = 100) {
    cout << "Book Title: " << title << endl;
    cout << "Author: " << author << endl;
    cout << "Number of Pages: " << pages << endl;
    cout << "----------------------------" << endl;
}

int main() {
bookDetails("C++ Programming", "Bjarne Stroustrup", 350); // Calling function with all arguments
bookDetails("Python Basics");     // Calling function with only title (default author & pages used)
bookDetails("Java Fundamentals", "James Gosling");   // Calling function with title and author (default pages used)
    return 0;
}
```

```
Book Title: C++ Programming
Author: Bjarne Stroustrup
Number of Pages: 350
----------------------------
Book Title: Python Basics
Author: Unknown
Number of Pages: 100
----------------------------
Book Title: Java Fundamenta
Author: James Gosling
Number of Pages: 100
----------------------------
```

Passing object as an argument

```
#include <iostream>
using namespace std;
class Student {
  public:
   double marks;
   Student(double m) {
  marks = m;
   }
};
// function that has objects as parameters
double average_marks(Student s1, Student s2) {
    s1.marks=s1.marks+10;
  return (s1.marks + s2.marks)/2 ; // return the average of marks of s1 and s2
}
int main() {
   Student student1(88.0), student2(56.0);
 // pass the objects as arguments
  cout << "Average Marks = " << average_marks(student1, student2) << "\n";
 cout << "student 1 marks " <<student1.marks << "\n";
   return 0;
}
```

```
Average Marks = 77
student 1 marks 88
```

```
#include <iostream>
using namespace std;
class Student {
public:
   double marks;
   Student(double m){
     marks=m;
   }
};

// function that has objects as parameters
double average_marks(Student *s1, Student *s2) {
   s1->marks=s1->marks+10;
   return (s1->marks + s2->marks)/2 ; // return the average
}

int main() {
   Student student1(88.0), student2(56.0);
cout << "Average Marks = "
       << average_marks(&student1, &student2) << "\n";
 cout << "student 1 marks " <<student1.marks << "\n";
return 0;
}
```

```
Average Marks = 77
student 1 marks 98
```

Returning object from a function

```
#include <iostream>
using namespace std;

class Student {
  public:
   double marks1, marks2;
   void display()
{
   cout << "Inside member function\n";
   cout << "Marks 1 = " << marks1 << endl;
   cout << "Marks 2 = " << marks2 << endl;
}

};
```

```
// function that returns object of Student
Student createStudent() {
   Student tempstudent;

   // Initialize member variables of Student
   tempstudent.marks1 = 96.5;
   tempstudent.marks2 = 75.0;

   return tempstudent;
}
```

```
int main() {
   Student student1;

   // Call function
   student1 = createStudent();
   student1.display();
   return 0;
}
```

```
Inside member function
Marks 1 = 96.5
Marks 2 = 75
```

Friend Function

- A friend Function is a non-member function that is declared inside a class using the "friend" keyword, it has special access to the class's private and protected members.
- Since it's not a member it is not bound to a specific object, can’tbe overloaded based on objects, not use this pointer, and cannot be inherited by derived classes.

```
#include <iostream>
using namespace std;
class Student {
private:
   string name;
   int marks;
public:
   void readDetails(string, int);  // Declaring member function
   friend void showDetails(Student s);  // Declaring friend function
};
void Student :: readDetails(string n, int m) {     // Member function definition using scope resolution operator ::
      name = n;
      marks = m;
   }
void showDetails(Student s) {       // Friend function definition
   cout << "Student Name: " << s.name << endl;
   cout << "Marks: " << s.marks << endl;
}
int main() {
   Student stu;  // Creating a student object
   stu.readDetails("Anu",90);
   showDetails(stu);  // Using the friend function to display student details
   return 0;
}
```

```
#include <iostream>
using namespace std;
class Book {
private:
   string title;
   int pages;
public:
   Book(string t, int p) {     // Constructor to initialize book details
      title = t;
      pages = p;
   }
friend void compareBooks(Book b1, Book b2); // Declaring friend function
};

// Friend function definition (Can access private members of Book)
void compareBooks(Book b1, Book b2) {
   cout << "Comparing Books..." << endl;
   if (b1.pages > b2.pages)
      cout << b1.title << " has more pages than " << b2.title << "." << endl;
   else if (b1.pages < b2.pages)
      cout << b2.title << " has more pages than " << b1.title << "." << endl;
   else
      cout << "Both books have the same number of pages." << endl;
}
```

```
int main() {
   // Creating book objects
   Book book1("C++ Programming", 350);
   Book book2("Java Fundamentals", 400);

   // Using the friend function to compare books
   compareBooks(book1, book2);

   return 0;
}
```

```
// Add members of two different classes using friend functions
#include <iostream>
using namespace std;

// forward declaration
class ClassB;

class ClassA {
   int numA;
   public:
      // constructor to initialize numA to 12
      ClassA() { numA=12;}

       // friend function declaration
       friend int add(ClassA, ClassB);
};

class ClassB {
  int numB;
   public:
      // constructor to initialize numB to 1
      ClassB() { numB=1;}

      // friend function declaration
      friend int add(ClassA, ClassB);
};
```

```
// access members of both classes
int add(ClassA objectA, ClassB objectB) {
   return (objectA.numA + objectB.numB);
}

int main() {
   ClassA objectA;
   ClassB objectB;
   cout << "Sum: " << add(objectA, objectB);
   return 0;
}
```

Friend Class

- When a class is declared a friend class, all the member functions of the friend class become friend functions.

```
class ClassB;

class ClassA {
   // ClassB is a friend class of ClassA
   friend class ClassB;
   ... .. ...
}

class ClassB {
   ... .. ...
}
```

Since ClassB is a friend class, we can access all members of ClassA from inside ClassB.

However, we cannot access members of ClassB from inside ClassA.

```
// C++ program to demonstrate the working of friend class

#include <iostream>
using namespace std;

// forward declaration
class ClassB;

class ClassA {
   private:
      int numA;

      // friend class declaration
      friend class ClassB;

   public:
      // constructor to initialize numA to 12
      ClassA() { numA = 12; }
  void printA(){
cout << "numA is " << numA << endl;
}
};

class ClassB {
  int numB;
  public:
      // constructor to initialize numB to 1
      ClassB(){ numB = 1; }

   // member function to add numA
   // from ClassA and numB from ClassB
   int add() {
      ClassA objectA;
      objectA.printA();
      cout << "numB is " <<numB<< endl;
      return objectA.numA + numB;
   }
};
int main() {
   ClassB objectB;
      cout << objectB.add();
   return 0;
}
```

```
numA is 12
numB is 1
13
```

Lambda Functions

- A **lambda function** is a small, anonymous function we can write directly in our code. It's useful when we need a quick function without naming it or declaring it separately.
- <u>Syntax</u>

```
[capture](parameters) -> return_type {
// function body
};
```

Capture clause [ ] → Specifies which external variables the lambda can access.
Parameters ( ) → Like normal function parameters.
Return type -> type → Optional (compiler can usually deduce it).
Body { } → Function code.

Simple Example

```
#include <iostream>
using namespace std;

int main() {
    auto greet = []() {
        cout << "Hello, Lambda!";
    };

    greet();   // calling lambda
    return 0;
}
```

```
#include <iostream>
using namespace std;
int main() {
  string x="C++";
  int y=10;
   auto show = [=]() {
    cout << x << endl;
    return y;
  };
  cout << show();
  return 0;
}
```

Every lambda expression has a unique, unnamed compiler-generated type. Because you cannot manually type out this internal name, auto is the standard way to declare and store a lambda in a variable.

Lambda with Parameters

```
#include <iostream>
using namespace std;

int main() {
      auto add = [](int a, int b) {
       return a + b;
   };
   int x,y;
   cin >> x >> y;
   cout << add(x, y);
   return 0;
}
```

Lambda with Capture

Capture by Value

```
#include <iostream>
using namespace std;
int main() {
  string x="C++";
  auto show = [=]() {
    cout << x << endl;
    //x="C#"; Error – Read-only
    cout << x << endl;
  };
  cout << x << endl;
  x="JAVA";
  show();
  cout << x << endl;
  return 0;
}
```

```
C++
C++
C++
JAVA
```

Capture by Reference

```
#include <iostream>
using namespace std;
int main() {
  string x="C++";
  auto show = [&]() {
    cout << x << endl;
    x="C#";
    cout << x << endl;
  };
  cout << x << endl;
  x="JAVA";
  show();
  cout << x << endl;
  return 0;
}
```

```
C++
JAVA
C#
C#
```

Lambda with explicit return type

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
   auto divide = [](double a, double b)->double {
      if (b == 0) {
         cout << "Division by zero not allowed!" << endl;
         return 0.0;
      }
      return a / b;
   };
   double result = divide(10.0, 2.0);
   cout << fixed << setprecision(2);
   cout << "Result: " << double (result) << endl;
   return 0;
}
```

Const member functions

- functions that are denied permission to change the values of the data members of their class.
- *return_type function_name*() **const**;
- When a function is declared as const, it can be called on any type of object, const object as well as non-const objects.
- Whenever an object is declared as const, it needs to be initialized at the time of declaration. however, the object initialization while declaring is possible only with the help of constructors.

```
#include <iostream>
using namespace std;

class Test {
   int value;

public:
   Test(int v = 0) { value = v; }

   // const member function
   int getValue() const { return value; }
};

int main()
{
   // non const object
   Test t(20);
   cout << t.getValue();
   return 0;
}
```